

DATA ANALYTICS

FASE 3 | AULA 01 -

INTRODUÇÃO AO BANCO DE DADOS RELACIONAIS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	18
O QUE VOCÊ VIU NESTA AULA?	21
REFERÊNCIAS	22

EMANIP

O QUE VEM POR AÍ?

Bem-vindo e bem-vinda à primeira aula sobre Bancos de Dados Relacionais! Nesta jornada, você terá a oportunidade de entender o que são os Bancos de Dados e porque os Bancos de Dados Relacionais são tão importantes no mundo da tecnologia e dos negócios. Pense em quantas vezes você já usou um aplicativo ou um site que exigiu o armazenamento e o acesso rápido a informações, tais como seu nome, endereço ou histórico de compras. Por trás dessas operações, há uma estrutura organizada que garante que tudo isso funcione de maneira eficiente: o banco de dados relacional.

Ao longo desta aula, vamos explorar os fundamentos que sustentam esses sistemas, como tabelas, chaves primárias, chaves estrangeiras e relacionamentos entre dados. Você aprenderá como essas estruturas são usadas para armazenar, organizar e recuperar informações de forma lógica e eficiente. Vamos desconstruir os conceitos básicos para que você possa, ao final desta aula, ter uma visão clara de como essas tecnologias se conectam ao nosso dia a dia e como elas podem ser aplicadas na prática. Prontos e prontas para começar?

HANDS ON

Nesta série de videoaulas, você terá a oportunidade de criar seu primeiro banco de dados relacional utilizando o Amazon RDS com o PostgreSQL. Ao longo dos vídeos, vamos guiá-lo(a) passo a passo, desde a configuração da instância no AWS até a execução de comandos SQL para manipulação de dados. Este hands-on não apenas fortalecerá sua compreensão dos conceitos teóricos apresentados nos materiais, mas também lhe proporcionará a experiência prática necessária para operar com os bancos de dados na nuvem.

Na próxima seção, você encontrará os comandos e instruções que utilizaremos durante o hands-on.

SAIBA MAIS

Instruções e Comandos:

Acessando o AWS Management Console:

- Faça login na sua conta da AWS e acesse o AWS Management Console.

Criando uma Instância do RDS:

- Navegue até o serviço RDS.
- Clique em “**Create Database**” e escolha “**Standard Create**”.
- Selecione PostgreSQL como o mecanismo do banco de dados, escolha a versão e configure a instância (tamanho e armazenamento).
- Configure as credenciais (usuário e senha) e clique em “**Create Database**”.

Conectando ao Banco de Dados:

- Após a instância ser criada, anote o Endpoint.
- Utilize um cliente SQL, como pgAdmin ou DBeaver, para se conectar ao banco de dados. Insira o Endpoint, nome do banco de dados, usuário e senha.

Criando uma Tabela

Após conectar-se, execute:

```
CREATE TABLE clientes (  
    id SERIAL PRIMARY KEY,  
    nome VARCHAR(100),  
    email VARCHAR(100)  
);
```

Código-fonte 1 – Criando uma Tabela
Fonte: elaborado pelo autor (2025)

Insira dados na tabela com:

```
INSERT INTO clientes (nome, email) VALUES  
  
('João da Silva', 'joao.silva@email.com'),  
  
('Maria Oliveira', 'maria.oliveira@email.com');
```

Código-fonte 2 – Inserindo Dados
Fonte: elaborado pelo autor (2025)

Verifique os dados com:

```
SELECT * FROM clientes;
```

Código-fonte 2 – Inserindo Dados
Fonte: elaborado pelo autor (2025)

Finalizando a Sessão:

- Desconecte-se do cliente SQL e gerencie suas instâncias no AWS Management Console.
- Os vídeos que acompanham estas instruções fornecerão uma demonstração prática de cada etapa, garantindo que você possa seguir junto e aplicar os conceitos aprendidos. Vamos começar!!!

O que são Bancos de Dados Relacionais?

Os bancos de dados relacionais (BDRs) são sistemas que armazenam dados de maneira estruturada, utilizando tabelas inter-relacionadas. Esse modelo foi introduzido na década de 1970 por Edgar F. Codd, que propôs a teoria relacional para organizar dados em forma de tabelas (ou "relações"). A principal característica dos BDRs é a forma como os dados são organizados e acessados, permitindo consultas complexas e a integridade referencial.

Exemplo: imagine uma tabela chamada "clientes" que armazena informações sobre os clientes de uma empresa. Cada linha da tabela representa um cliente e cada coluna contém informações específicas, como nome, e-mail e telefone. Assim, em vez de guardar todos os dados em um único bloco de texto, organizamos tudo de maneira que cada informação tenha seu espaço definido.

Estrutura de um Banco de Dados Relacional

A estrutura básica de um banco de dados relacional inclui:

Tabelas: são a base do banco de dados, organizando dados em linhas (tuplas) e colunas (atributos). Cada tabela deve ter uma chave primária, que é um identificador único para cada registro.

Chaves

- **Chave Primária:** um campo (ou conjunto de campos) que identifica de forma exclusiva cada linha na tabela.
- **Chave Estrangeira:** um campo em uma tabela que se relaciona com a chave primária de outra tabela, permitindo a criação de vínculos entre dados.

Relacionamentos: os BDRs permitem que tabelas se relacionem entre si, o que é essencial para manter a integridade dos dados e realizar consultas complexas. Existem três tipos principais de relacionamentos:

- **Um-para-um (1:1):** um registro em uma tabela corresponde a um único registro em outra tabela.
- **Um-para-muitos (1:M):** um registro em uma tabela pode estar associado a vários registros em outra tabela.
- **Muitos-para-muitos (M:N):** vários registros em uma tabela podem estar associados a vários registros em outra tabela, geralmente implementado com uma tabela intermediária.

Vantagens dos Bancos de Dados Relacionais

Os BDRs oferecem diversas vantagens, incluindo:

- **Estrutura Clara:** a organização em tabelas facilita a visualização e o gerenciamento de dados.
- **Integridade dos Dados:** com a definição de chaves primárias e estrangeiras, é mais fácil manter a integridade referencial, evitando dados duplicados ou inconsistentes.

- **Facilidade de Consulta:** a linguagem SQL (Structured Query Language) permite realizar consultas complexas de forma simples e eficiente.
- **Escalabilidade:** os BDRs podem ser facilmente escalados para suportar grandes volumes de dados.

Linguagem SQL

SQL é a linguagem padrão para manipulação de bancos de dados relacionais. Ela permite a execução de diversas operações, como:

Criação de Tabelas: define a estrutura do banco de dados.

```
CREATE TABLE clientes (  
    id SERIAL PRIMARY KEY,  
    nome VARCHAR(100),  
    email VARCHAR(100)  
);
```

Inserção de Dados: adiciona novos registros à tabela.

```
INSERT INTO clientes (nome, email)  
VALUES ('João da Silva', 'joao.silva@email.com');
```

Consultas: recupera informações específicas.

```
SELECT * FROM clientes WHERE nome = 'João da Silva';
```

A seguir estão alguns exemplos que ilustram a aplicação de conceitos de bancos de dados relacionais:

Tabela de Clientes

ID	Nome	Email
1	João da Silva	joao.silva@email.com
2	Maria Oliveira	maria.oliveira@email.com

Tabela 1 – Tabela de clientes
Fonte: elaborado pelo autor (2025)

Tabela de Pedidos

ID	Cliente_ID	Data	Valor
1	1	2024-10-01	100.00
2	2	2024-10-02	150.00

Tabela 2 – Tabela de pedidos
Fonte: elaborado pelo autor (2025)

Comandos SQL mais usados

Comando	Descrição	Exemplo
CREATE	Cria uma tabela ou banco de dados.	CREATE TABLE clientes (id SERIAL PRIMARY KEY, nome VARCHAR(100), email VARCHAR(100));
INSERT	Inserir novos registros em uma tabela.	INSERT INTO clientes (nome, email) VALUES ('João da Silva', 'joao.silva@email.com');
SELECT	Recupera dados de uma ou mais tabelas.	SELECT * FROM clientes; SELECT nome, email FROM clientes WHERE id = 1;
UPDATE	Atualiza dados existentes em uma tabela.	UPDATE clientes SET email = 'novo_email@email.com' WHERE id = 1;

DELETE	Remove registros de uma tabela.	DELETE FROM clientes WHERE id = 1;
ALTER	Modifica a estrutura de uma tabela existente.	ALTER TABLE clientes ADD telefone VARCHAR(15);
DROP	Remove uma tabela ou banco de dados.	DROP TABLE clientes; DROP DATABASE aula_banco_dados;
JOIN	Combina registros de duas ou mais tabelas.	SELECT clientes.nome, pedidos.data FROM clientes JOIN pedidos ON clientes.id = pedidos.cliente_id;
WHERE	Filtra os registros retornados por uma consulta.	SELECT * FROM clientes WHERE nome LIKE 'João%';
GROUP BY	Agrupa resultados com base em uma ou mais colunas.	SELECT COUNT(*) AS total_clientes FROM clientes GROUP BY nome;
ORDER BY	Ordena os resultados de uma consulta com base em uma ou mais colunas.	SELECT * FROM clientes ORDER BY nome ASC;
HAVING	Filtra grupos criados por uma cláusula GROUP BY.	SELECT nome, COUNT(*) FROM clientes GROUP BY nome HAVING COUNT(*) > 1;
LIMIT	Restringe o número de registros retornados em uma consulta.	SELECT * FROM clientes LIMIT 5;
DISTINCT	Remove duplicatas dos resultados retornados.	SELECT DISTINCT nome FROM clientes;
INDEX	Cria um índice para melhorar a performance de consultas em uma tabela.	CREATE INDEX idx_nome ON clientes (nome);

Tabela 3 – Comandos SQL
Fonte: elaborado pelo autor (2025)

Normalização de Dados

A normalização é um processo utilizado para organizar dados em um banco de dados, minimizando redundâncias e evitando anomalias de inserção, atualização e exclusão. O objetivo é dividir os dados em tabelas relacionadas de forma lógica. As formas normais mais comuns são:

Primeira Forma Normal (1NF): garante que todos os atributos de uma tabela sejam atômicos (não divisíveis) e que cada coluna tenha apenas um valor por linha. Também exige que cada linha seja única.

Segunda Forma Normal (2NF): exige que todos os atributos não chave sejam dependentes da chave primária. Isso significa que todos os dados em uma tabela devem estar relacionados à chave primária.

Terceira Forma Normal (3NF): remove dependências transitivas, ou seja, atributos que dependem de outros atributos não chave. Aqui, cada coluna deve depender diretamente da chave primária.

Exemplo: considere uma tabela de pedidos que contém o ID do pedido, o nome do cliente e o endereço de entrega. Se o nome do cliente e o endereço de entrega forem armazenados na mesma tabela que os pedidos, haverá duplicação de dados. Normalizando, podemos criar uma tabela separada para clientes e armazenar apenas uma referência ao cliente (ID) na tabela de pedidos.

Benefícios da Normalização

- Reduz a duplicação de dados e economiza espaço.
- Facilita a manutenção, pois atualizações são feitas em um único lugar.
- Melhora a consistência e a integridade dos dados.

Desvantagens da Normalização

- Pode reduzir a performance, pois são necessárias mais junções (joins) para recuperar dados relacionados.
- A complexidade do design aumenta conforme o nível de normalização avança.

Exemplo de Normalização de Dados

Vamos passar pelas três primeiras formas normais (1NF, 2NF e 3NF) usando um exemplo de um sistema de gerenciamento de pedidos de uma loja.

Tabela Inicial - Dados Não Normalizados

Imagine que temos uma tabela de pedidos onde todos os dados estão armazenados em uma única tabela. A estrutura é a seguinte:

Pedido_ID	Cliente_Nome	Cliente_Endereco	Produto_Nome	Quantidade	Preço_Total
1	João Silva	Rua A, 123, São Paulo	Caneca Personalizada	2	40
2	Maria Costa	Rua B, 456, Rio de Janeiro	Caderno A5	1	15
3	João Silva	Rua A, 123, São Paulo	Caderno A5	3	45

Tabela 4 – Dados não normalizados
Fonte: elaborado pelo autor (2025)

Nesta tabela, há várias redundâncias, como a repetição do endereço do cliente. Vamos normalizar esses dados passo a passo.

Primeira Forma Normal (1NF): a 1NF exige que os valores em cada coluna sejam atômicos (não divisíveis) e que não haja grupos de repetição. Na tabela inicial, já cumprimos a 1NF, pois cada coluna contém um único valor por linha.

Segunda Forma Normal (2NF): a 2NF elimina dependências parciais, onde atributos não chave devem depender inteiramente da chave primária. Precisamos dividir a tabela inicial em duas: **Clientes** e **Pedidos**.

Tabela Clientes

Pedido_ID	Cliente_Nome	Cliente_Endereco
1	João Silva	Rua A, 123, São Paulo
2	Maria Costa	Rua B, 456, Rio de Janeiro
3	João Silva	Rua A, 123, São Paulo

Tabela 5 – Tabela clientes
Fonte: elaborado pelo autor (2025)

Tabela Pedidos

Pedido_ID	Cliente_ID	Produto_Nome	Quantidade	Preço_Total
1	1	Caneca Personalizada	2	40
2	2	Caderno A5	1	15
3	1	Caderno A5	3	45

Tabela 6 – Tabela pedidos
Fonte: elaborado pelo autor (2025)

Agora, cada cliente tem um identificador único e os pedidos referenciam o Cliente_ID. Isso remove a redundância dos dados de cliente.

Terceira Forma Normal (3NF): a 3NF elimina dependências transitivas, onde um atributo não chave depende de outro atributo não chave. Vamos criar uma tabela separada para produtos.

Tabela Produtos

Produto_ID	Produto_Nome	Preço_Unitário
1	Caneca Personalizada	20.00
2	Caderno A5	15.00

Tabela 7 – Tabela produtos
Fonte: elaborado pelo autor (2025)

Tabela Pedidos (atualizada)

Pedido_ID	Cliente_ID	Produto_ID	Quantidade	Preço_Total
1	1	1	2	40
2	2	2	1	15
3	1	2	3	45

Tabela 8 – Tabela pedidos (atualizada)
Fonte: elaborado pelo autor (2025)

Dessa forma, eliminamos a dependência dos produtos dentro da tabela de pedidos, tornando os dados mais organizados e fáceis de manter.

Após a normalização, temos três tabelas organizadas e sem redundâncias:

- **Clientes:** contém informações sobre os clientes.
- **Produtos:** contém informações sobre os produtos.
- **Pedidos:** relaciona clientes e produtos, registrando detalhes da compra.

Este processo de normalização garante que os dados estejam organizados de maneira eficiente, evitando redundâncias e facilitando a manutenção e consulta dos dados no banco de dados.

Integridade Referencial

A integridade referencial garante que as relações entre tabelas sejam mantidas corretamente. Isso é obtido pelo uso de chaves estrangeiras (foreign keys), que asseguram que um valor em uma coluna seja consistente com os valores da tabela referenciada.

Exemplo: se tivermos uma tabela de clientes e outra de pedidos, onde cada pedido está associado a um cliente, a chave estrangeira em "pedidos" deve referenciar a chave primária da tabela "clientes". Isso evita, por exemplo, que um pedido seja inserido com um cliente que não existe no sistema.

Transações e Controle de Concorrência

Transações

Uma transação é um conjunto de operações que são executadas como uma única unidade de trabalho. Elas garantem que, mesmo em cenários de falha, o banco de dados permaneça consistente. Transações seguem o modelo ACID:

Atomicidade: todas as operações em uma transação são executadas completamente ou então nenhuma delas é executada.

Consistência: o banco de dados muda de um estado consistente para outro estado consistente após uma transação.

Isolamento: transações concorrentes não interferem em outras.

Durabilidade: uma vez que uma transação é concluída, as mudanças são permanentes, mesmo que ocorra uma falha no sistema.

Controle de Concorrência

Em ambientes com múltiplos usuários, o controle de concorrência é necessário para garantir que transações concorrentes não corrompam os dados. Técnicas comuns incluem:

Bloqueios (Locks): permite ou impede acesso simultâneo a um recurso ou linha de dados.

MVCC (Multi-Version Concurrency Control): o PostgreSQL usa essa técnica para manter várias versões dos dados e garantir que as leituras não sejam bloqueadas por escritas.

Otimização de Consultas e Índices

Com o aumento do volume de dados, a performance das consultas torna-se crucial. A seguir, exploraremos algumas técnicas de otimização:

Índices: Índices são estruturas de dados que melhoram a velocidade das consultas, mas que consomem recursos de armazenamento e afetam o tempo de inserção e atualização. O PostgreSQL, por exemplo, suporta diversos tipos de índices:

- **B-Tree:** padrão para a maioria das colunas.

- **Hash:** útil para consultas de igualdade.
- **GIN e GiST:** usados para buscas de texto completo e tipos de dados complexos.

Exemplo:

```
CREATE INDEX idx_cliente_nome ON clientes (nome);
```

Este índice permite buscas mais rápidas pelo nome dos clientes.

Análise de Consultas com EXPLAIN: o comando EXPLAIN no PostgreSQL ajuda a identificar possíveis gargalos em uma consulta, mostrando como o banco de dados planeja executar a query. Ao utilizar o EXPLAIN, é possível visualizar a ordem de execução, os tipos de junção utilizados e o custo estimado.

Exemplo:

```
EXPLAIN SELECT * FROM clientes WHERE nome = 'João';
```

Modelagem Avançada de Dados: conforme o sistema cresce, o design inicial do banco de dados pode precisar ser adaptado para suportar novos requisitos e performance. Técnicas como particionamento, replicação e sharding são utilizadas para escalar o banco de dados.

- **Particionamento:** divisão lógica de tabelas grandes em partes menores para melhorar a performance de consultas.
- **Replicação:** cópias de bancos de dados para garantir alta disponibilidade e recuperação de falhas.
- **Sharding:** divisão física do banco de dados para distribuir a carga de trabalho.

Desafios e Limitações dos Bancos de Dados Relacionais

Apesar de serem amplamente usados, os bancos de dados relacionais possuem limitações:

Escalabilidade Horizontal: enquanto bancos de dados NoSQL escalam horizontalmente com facilidade, bancos relacionais geralmente requerem soluções mais complexas para se adaptarem a essa necessidade.

Complexidade de Modelagem: à medida que os sistemas crescem, a modelagem e manutenção do esquema se tornam mais complexas e podem necessitar de revisões frequentes.

Exercícios Práticos para Fixação

Exercício 1: normalizando tabelas

Crie duas tabelas: uma para "clientes" e outra para "pedidos", garantindo que as colunas sejam normalizadas até a 3ª forma normal.

Exercício 2: criando índices e otimizando consultas

Crie um índice em uma tabela e execute uma consulta usando EXPLAIN para visualizar como o índice impacta a performance.

MERCADO, CASES E TENDÊNCIAS

A Evolução dos Bancos de Dados Relacionais no Mercado

Nos últimos anos, os bancos de dados relacionais têm se mantido como a espinha dorsal de muitas organizações que lidam com grandes volumes de dados. A capacidade de fornecer consistência e integridade dos dados, aliada a ferramentas SQL avançadas para análise, faz com que essas tecnologias continuem a ser amplamente utilizadas. O surgimento de bancos de dados relacionais em ambientes de nuvem, como o [Amazon RDS](#), [Azure SQL Database](#) e [Google Cloud SQL](#), permitiu às empresas escalarem suas operações e aumentarem a disponibilidade sem investir pesadamente em infraestrutura física.

Case de Sucesso: Netflix e o Uso de Bancos de Dados Relacionais

A Netflix, uma das maiores plataformas de streaming do mundo, é um exemplo clássico de como bancos de dados relacionais podem ser usados de forma eficiente. Apesar de utilizar também tecnologias NoSQL para certas operações, a empresa ainda depende fortemente de bancos de dados relacionais para gerenciar suas operações financeiras e de faturamento. Com o uso de PostgreSQL em ambientes de nuvem, a Netflix conseguiu garantir escalabilidade e segurança para operações críticas, consolidando sua posição no mercado. Mais sobre o uso de PostgreSQL pode ser encontrado [aqui](#).

O Futuro dos Bancos de Dados Relacionais

As tendências mostram que os bancos de dados relacionais estão evoluindo para se adaptar ao contexto de Big Data e análise avançada. Hoje, tecnologias como o [Aurora da AWS](#) e o [BigQuery](#), que suportam SQL, permitem consultas altamente paralelas e escaláveis, combinando o melhor dos dois mundos: a integridade dos bancos de dados relacionais com a flexibilidade dos sistemas analíticos modernos. A combinação de SQL com ferramentas de Machine Learning é uma das principais tendências, possibilitando a análise preditiva diretamente no banco de dados.

Publicações Recentes

Nos últimos anos, muitos artigos têm sido publicados sobre a integração de bancos de dados relacionais com a tecnologia de **Inteligência Artificial (IA)**. Uma pesquisa recente publicada no [Journal of Database Management](#) mostrou como algoritmos de IA podem ser integrados com consultas SQL para otimizar o desempenho de bancos de dados.

Inteligência Artificial e Automação no Mundo SQL

Uma das tendências emergentes é o uso da **IA para otimização automática de consultas SQL**. Ferramentas como o AI SQL Tuning Advisor, da Oracle, já estão sendo desenvolvidas para analisar automaticamente consultas SQL e sugerir otimizações, reduzindo o tempo de resposta e aumentando a eficiência. Esse movimento promete diminuir o esforço dos DBAs e de pessoas desenvolvedoras, deixando as plataformas mais "inteligentes" na administração dos dados. Saiba mais sobre otimização aqui: [SQL](#).

Ferramentas Populares no Mercado

No mercado atual de bancos de dados relacionais, destacamos algumas ferramentas, dentre elas:

[MySQL](#): Open-source e amplamente utilizado para aplicações web.

[PostgreSQL](#): conhecido pela sua robustez e recursos avançados de SQL. Informações adicionais.

[Microsoft SQL Server](#): forte integração com o ecossistema da Microsoft.

[Amazon Aurora](#): compatível com MySQL e PostgreSQL, com alta disponibilidade em nuvem.

[Snowflake](#): embora seja um data warehouse, suporta SQL e está cada vez mais ganhando espaço no mercado de análise de dados.

Como Startups Estão Utilizando SQL para Crescer

Várias startups estão adotando bancos de dados relacionais para validar rapidamente hipóteses de negócio e armazenar dados de forma confiável. Um exemplo é a startup [Zenvia](#), que utiliza PostgreSQL como base para gerenciar

milhões de mensagens enviadas diariamente por sua plataforma de comunicação, garantindo a segurança e a integridade dos dados dos clientes.

EXEMPLO

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você foi introduzido(a) ao mundo dos bancos de dados relacionais. Exploramos conceitos fundamentais, tais como: o que é um banco de dados, quais os principais tipos e características dos bancos de dados relacionais, além de entender como os dados são organizados em tabelas e relacionados entre si por meio de chaves primárias e estrangeiras.

Discutimos os conceitos de SQL e os comandos básicos, como **CREATE**, **INSERT**, **SELECT**, **UPDATE** e **DELETE**, que permitem a manipulação e consulta dos dados. Também abordamos a importância da integridade dos dados e demos os primeiros passos para entender como projetar um banco de dados relacional eficiente.

REFERÊNCIAS

HEUSER, Carlos A. **Projeto de Banco de Dados**. Porto Alegre: Bookman, 2009.

LAUDON, K. C.; LAUDON, J. P. **Sistemas de Informação Gerenciais**. São Paulo: Pearson, 2013.

LEMOES, C.; CASTRO, V. **Modelagem de Dados: Conceitual e Lógica**. Rio de Janeiro: Elsevier, 2015.

MACHADO, F. N.; MAIA, M. E. **Banco de Dados: Projeto e Implementação**. São Paulo: Érica, 2009.

NETFLIX. **How Netflix Uses Amazon RDS for Scalable Databases**. Amazon Web Services. 2020. Disponível em: <https://aws.amazon.com/solutions/case-studies/netflix/>. Acesso em: 26 mar.2025.

NIELD, Thomas. **Introdução à Linguagem SQL: Abordagem Prática Para Iniciantes**. São Paulo: Alta Books, 2019.

OLIVEIRA, J. L. **Técnicas de Modelagem de Dados para Banco de Dados Relacionais**. Revista Tecnologia e Sistemas, Edição 12, 2020, p. 32-45. Disponível em: <http://www.revistatecnologia.com/artigos/modelagem-relacional>. Acesso em: 26 mar. 2025.

SILBERSCHATZ, A.; KORTH, H.; SUDARSHAN, S. **Sistema de Banco de Dados**. Rio de Janeiro: Elsevier, 2011.

TANIMURA, Cathy. **SQL Para Análise de Dados**. São Paulo: Novatec, 2022.

PALAVRAS-CHAVE

Bancos de Dados. SQL. Modelagem de Dados.

EMANIP

The background is a dark blue field filled with numerous small, light blue dots, resembling a starry sky or a data visualization. Overlaid on this are several large, wavy, translucent lines in shades of blue and yellow, creating a sense of motion and depth. Scattered throughout are various geometric shapes: a thin vertical line, a circle containing the number '7', a small circle, a cross, a small circle, and a hexagon.

POSTECH